Feed Generation

candidates, rank, blend, paginate, cache

Watch the 4-min video

In one line: a feed is a funnel: gather a few thousand **candidates** cheaply, **rank** them with a model that predicts engagement, **blend** with rules for diversity, ads and already-seen items, then serve a page with a **cursor**, using caches that hold IDs rather than full posts.

1 Twenty items out of thousands

You open a photo app and within about 200 ms see a feed: friends' posts, a suggested reel, an ad, in a chosen order. The system picked those 20 items out of thousands of eligible ones, just for you. It cannot run an expensive model over millions of posts per request, so it works in stages, each doing more work per item on fewer items.

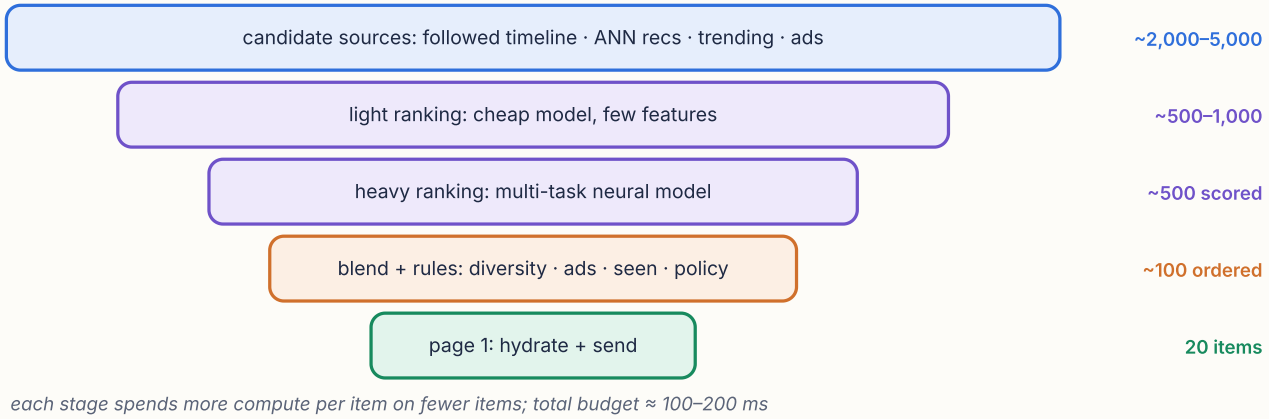


Figure 1. The ranking funnel. Numbers are typical orders of magnitude, not fixed values.

2 Candidate generation: recall first

Candidates come from several **sources** in parallel, each with a quota:

- **Followed accounts:** the user's precomputed timeline of post IDs (built by fan-out, lesson 30).
- **Recommendations:** embedding retrieval: a two-tower model maps users and posts into one vector space, and an ANN index returns the nearest posts (lessons 22–23). Also "friends engaged with this" and topic sources.
- **Trending and fresh** content (lesson 40), and **ads**, which come from a separate auction and are inserted later.

This stage optimises **recall**: a great post that never becomes a candidate cannot be ranked up. It must be cheap: cache lookups and ANN queries, a few tens of milliseconds, with eligibility filters (blocked users, deleted posts, age limits) applied early.

3 Ranking: predicting engagement

A **light ranker** (small model, few features) trims the set; then the **heavy ranker**, usually a multi-task neural network, predicts several probabilities per candidate: like, comment, share, watch to the end, and negative signals such as hide or report. A **value function** combines them, for example `score =`

$w_1 \cdot P(\text{like}) + w_2 \cdot P(\text{comment}) + w_3 \cdot P(\text{share}) - w_4 \cdot P(\text{hide})$; the weights encode what the product wants to encourage.

Features: who posted (your interaction history with them, the **affinity**), content type, the post's age and early engagement, and your context (device, time, session). They are read from a low-latency **feature store**. Models are trained on logged impressions ("shown at position 3, liked") and retrained regularly as behaviour changes, so every served feed must log what was shown.

4 Blending and rules

Pure score order produces bad feeds: five posts from one prolific friend in a row, three ads together, a post you saw yesterday. A re-ranking pass applies rules:

- **Diversity:** at most one item per author in any window of N positions; mix content types (photo, video, reel).
- **Ads and inserts:** fixed or min-gap slots (e.g. an ad no closer than every 5th position).
- **Seen filter:** drop items already shown; a per-user seen set or Bloom filter (lesson 39) makes this cheap.
- **Integrity:** remove items that break policy or were reported, and demote low-quality content.

5 Paginating a feed that keeps changing

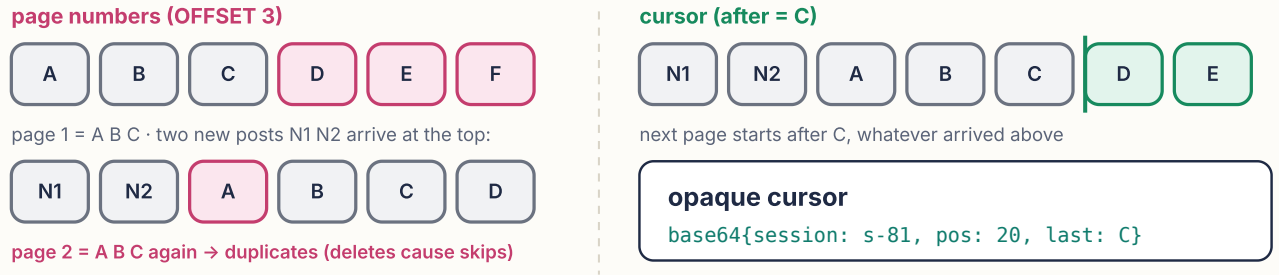


Figure 2. Offsets count from the top, which moves. A cursor remembers **where** you stopped, so new posts never cause duplicates or skips.

For a **chronological** feed, the cursor is a keyset: `(created_at, post_id)` of the last item, and the next query is "older than this". For a **ranked** feed there is no natural sort key, so the server keeps the ranked list for this session (e.g. 100 IDs in a cache with a short TTL) and the cursor points into it, plus the seen set to exclude repeats. Make the cursor **opaque** (encoded, maybe signed) so clients cannot depend on its contents.

6 Caching for a fast open

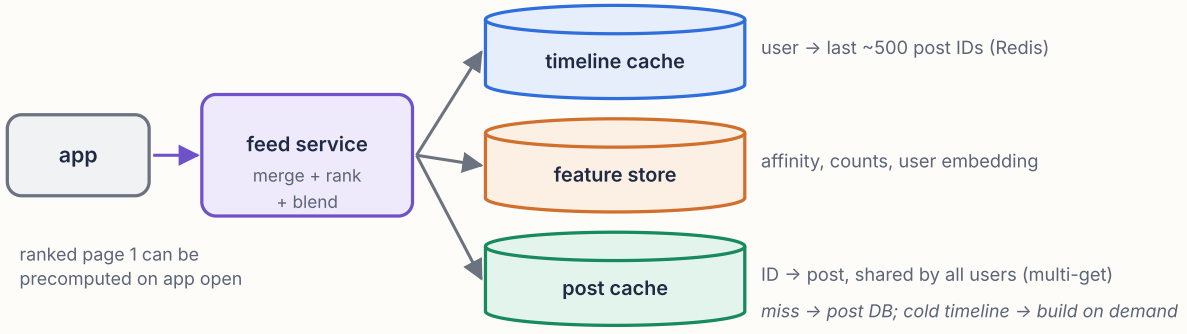


Figure 3. The read path. Timelines hold IDs only; post bodies come from one shared cache, so an edit or delete updates everyone at once.

- **Timeline cache** per user: a list of the latest few hundred post IDs (a few KB per user), not post objects. Storing full posts per follower would multiply storage and make edits impossible.
- **Post cache** shared by all users: hydrate a page with one multi-get of 20 IDs. Counters (likes) are often fetched separately because they change constantly (lesson 38).
- **Precompute** the first ranked page when signals suggest the user will open the app soon, so it appears instantly.
- **Cold cache** (inactive user): build the feed on demand from the followed accounts' recent posts. Slower, but correct.

Stage	Items in → out	Typical budget	Optimises
Candidate generation	millions → thousands	20–50 ms	recall, cheapness
Light + heavy ranking	thousands → hundreds scored	50–100 ms	precision of order
Blending / rules	hundreds → ordered list	few ms	feed quality, ads, policy
Hydration	20 IDs → 20 posts	~10 ms	latency (cache hits)

WORKED EXAMPLE: AN INSTAGRAM-STYLE HOME FEED

The user opens the app. Candidates: 600 IDs from their followed-accounts timeline, 300 recommended reels from embedding retrieval, 100 trending posts, and eligible ads from the auction. After filters (blocked, deleted, seen) about 1,000 remain. The heavy model scores them for P(like), P(comment), P(share) and P(hide); the value function orders them. Blending puts a reel every few posts, an ad every few more, and never two posts from the same account in a row. The top 20 IDs are hydrated from the post cache and returned with cursor `{session: s-81, pos: 20}`; the remaining ranked list stays cached for a few minutes, so scrolling serves positions 21–40 from it without re-ranking, and impressions are logged for the next training run.

TRADE-OFFS

- More candidates improve recall but cost latency and ranking compute.
- Heavier models rank better but need stricter funnels or precomputation.
- Session-cached rankings make pagination stable but slightly stale; re-ranking on scroll is fresher but can reorder what the user saw.
- Optimising pure engagement can favour clickbait; negative signals and diversity rules counterbalance it.

COMMON MISTAKES

- Running the expensive model on everything instead of on a small candidate set.
- Offset pagination on a feed that changes, causing duplicates and skips.
- Caching full post objects inside every user's timeline.
- Not logging impressions, so the model has no labels to learn from.

7 Key terms

Candidate generation: cheap, high-recall gathering of items to rank

Ranking model: predicts engagement probabilities per candidate

Value function: weighted mix of predictions into one score

Blending: rule-based re-ranking for diversity, ads and policy

Cursor: opaque token marking where the next page starts

Hydration: turning post IDs into full posts via a shared cache

8 Interview questions

Q1. How do you build a personalised feed? BASIC

Generate candidates (follows, recommendations, ads), rank them with an engagement model, blend with rules for diversity, ads and seen items, and cache timelines and post details.

Q2. How do you paginate a changing feed? BASIC

Use cursors, not page numbers: a keyset for chronological feeds, or a pointer into a cached ranked list for ranked feeds.

Q3. The feed takes 900 ms. Where do you look? SCENARIO

Profile per stage: candidate fan-in (too many sources or cache misses), ranking (too many candidates for the heavy model), and hydration (N+1 lookups instead of multi-get); shrink the funnel and precompute.

Q4. Users complain they see the same post every time they open the app. SCENARIO

Add a seen filter (per-user seen set or Bloom filter) in blending, and down-rank items with prior impressions.

Q5. Why can't ranking fix a post that's missing from the feed? TRICKY

Ranking only orders candidates; if retrieval never selected the post, no model sees it. Recall must be fixed at candidate generation.

Q6. Why store IDs rather than posts in each timeline? TRICKY

IDs are tiny and edits or deletes then apply everywhere through one shared post cache; copies per follower would explode storage and go stale.

Next: SD-Q-005 · Fan-Out on Write vs Read backendroadmap.com